

## Reviewer 1

**Novelty:** 8 (clearly novel - major differences from all existing ideas)

**Rationale:** The use of semantic similarity to constrain CoT-styled generation is very new. I have not seen similar work on it.

**Feasibility:** 5 (Moderately feasible: It can probably be executed within the given time frame but would require careful planning, efficient use of APIs or some advanced computational strategies to overcome the limited GPU resources, and would require some modifications to the original proposal to make it work.)

**Rationale:** The pipeline is feasible to me. The major challenge would be finding the similarity threshold for each dataset.

**Expected Effectiveness:** 3 (Low Effectiveness: The idea might work in some special scenarios but you don't expect it to work in general.)

**Rationale:** I see some drawbacks in this pipeline. First, manually tuning the similarity threshold seems not the best practice for scalable applications. The GSM8K math dataset contains pretty elementary math problems. In that case, the semantic similarity threshold should be set very high, since these basic math concepts involved in the prompt and the CoT breakdown would be determined as highly similar by most existing embedding methods. This brings the question of whether this similarity threshold is non-trivial at all for some tasks.

**Excitement:** 6 (Learning positive: exciting enough to be accepted at a major AI conference, but still has some weaknesses or somewhat incremental)

**Rationale:** Constraining CoT breakdowns is a novel idea and deserves more work and exploration. While the use of semantic similarity has many drawbacks (such as tuning the threshold, task-sensitive, non-scalable), it can still show us some valuable results about constraining CoT breakdowns.

**Overall Score:** 5 (Decent idea but has some weaknesses or not exciting enough, marginally below the acceptance threshold of major AI conferences)

**Rationale:** There are some clear drawbacks inherent to the method, as discussed earlier. If the authors can overcome these limitations, this idea could yield some interesting findings useful for our understanding of CoT behavior and could pass above a major conference threshold.

**Confidence:** 3 (You are fairly confident that the evaluation is correct)

## Reviewer 2

**Novelty:** 4

**Rationale:** Generally this method is a way of rejection sampling to improve factuality. It is somewhat not too different from previous literature for "constrained decoding" for improving factuality: - Constrained Abstractive Summarization: Preserving Factual Consistency with Constrained Generation - Don't Say What You Don't Know: Improving the Consistency of Abstractive Summarization by Constraining Beam Search

**Feasibility:** 9

**Rationale:** Simple prompting approach that is easy to implement. Evaluation is simple.

**Expected Effectiveness:** 3 (Low Effectiveness: The idea might work in some special scenarios but you don't expect it to work in general.)

**Rationale:** 1. Right now most LLMs hallucinate in a subtle way: they say things in semantically correct or reasonable ways, but the precise fact is incorrect. Using semantic similarity as a measurement to gauge/control hallucination might not be able to solve the problem. 2. The rejection sampling is based on another LLM—what if the LLM also hallucinates?

**Excitement:** 3 (Mediocre: this idea makes marginal contributions and is very incremental)

**Rationale:** The method is not that novel and I think the method is not that effective and might not solve the problem at all.

**Overall Score:** 3 (Clear rejection for major AI conferences)

**Rationale:** The experiment design is kind of simple and the evaluation is not comprehensive. I think the idea is in the range of 4 but the experiment plan further reduces my score.

**Confidence:** 5 (You are absolutely certain that the evaluation is correct and very familiar with the relevant literature)

### Reviewer 3

**Novelty:** 3 (mostly not novel - you can find very similar ideas)

**Rationale:** The idea of extracting key semantic concepts, measuring the relevance of the candidate next step, and possibly rejecting/revising the step is very similar to incorporating self-critique into multi-step reasoning problems. Different versions of this are already commonly used, especially for solving math problems.

**Feasibility:** 8 (Highly Feasible: Straightforward to implement the idea and run all the experiments.)

**Rationale:** The proposed approach should be straightforward to implement: it only requires prompt engineering to extract semantic concepts and evaluate the relevance of a candidate next step.

**Expected Effectiveness:** 3 (Low Effectiveness: The idea might work in some special scenarios but you don't expect it to work in general.)

**Rationale:** Compared to chain-of-thought prompting, there's a reasonable chance this method could work better: it could help identify when a reasoning step becomes irrelevant to the original question. However, since such self-critique methods have already been explored, it's unlikely that this instantiation will work significantly better than previous ones. Also, the proposed idea of extracting relevant semantic concepts and measuring semantic similarity seems a bit vague, and it's not reflected in the provided examples.

**Excitement:** 2

**Rationale:** The proposed method is too similar to existing works; it doesn't contain novel insights that would meaningfully boost current LM performance or introduce new ideas worth building on. It would not be an exciting paper.

**Overall Score:** 2 (Strong rejection for major AI conferences)

**Rationale:** Similar to the reasoning above: the proposal is too similar to existing works, it doesn't introduce new ideas or insights, and is unlikely to meaningfully improve current LM performance.

**Confidence:** 4 (You are confident but not absolutely certain that the evaluation is correct)

## V Example Idea: Autoprompting: Generate Diverse Few-shot Examples for Any Application

### Autoprompting: Generate Diverse Few-Shot Examples for Any Application (Part 1)

**1. Problem Statement:** Adding natural language capabilities to existing software requires manually crafting few-shot prompts, which is tedious and does not guarantee high coverage.

**2. Motivation:** Integrating natural language capabilities into software applications often necessitates manually creating few-shot prompts, a process that is time-consuming and may not ensure comprehensive coverage. An "Autoprompting" system capable of automatically generating diverse and relevant few-shot examples tailored to specific applications would significantly reduce manual effort, improve coverage and versatility, and enable rapid prototyping and iteration of natural language capabilities. Large Language Models can iteratively test different functionalities of an application and make adjustments to few-shot prompts akin to a human developer. This approach would ultimately democratize the integration of such capabilities across a wide range of applications and industries.

**3. Proposed Method:** This method leverages a Large Language Model (LLM) with coding capabilities. It involves the following core steps:

1. Extract all user-facing functions and gather their documentation and unit tests, if available.
2. Generate diverse natural language prompts to utilize each function, defining the expected output.
3. Generate code from the natural language prompts and execute the corresponding functions.
4. If the code fails:
  - Update the code and retry
  - If the code runs but produces an incorrect result, update it using insights from unit tests or general reasoning.
5. Once you have a few exemplar prompts for all (or desired) functions, generate prompts that compose multiple functions together and repeat step 4.

By iteratively refining code generation from natural language and leveraging available documentation and tests, this process aims to create an LLM capable of correctly implementing functions based on natural language instructions.

#### 4. Step-by-Step Experiment Plan:

- **Applications:** When collecting applications from GitHub, prioritize those with clear, well-written documentation and comprehensive test suites. Include applications from different domains and with varying levels of complexity to ensure a diverse dataset.
- **Few shots and feasibility:** Create manual few-shot examples to understand the complexity of the functions and the quality of the documentation. Begin by creating 4-5 examples for any function, which could also serve as a starting point for the LLM.
- **Extract functions and metadata:** Utilize static code analysis tools to ensure accurate and comprehensive extraction of functions, documentation, and test cases. Consider extracting additional metadata, such as function signatures, dependencies, and comments, as they can provide valuable context.
- **NL Module:** Generate diverse user utterances and incorporate techniques to handle variations in natural language. For each user utterance, generate the expected outcome. Consider generating negative test cases to improve the model's ability to handle invalid or ambiguous inputs.
- **Execution Module:** Incorporate sandboxing or containerization techniques to ensure a secure and isolated execution environment when executing the generated code. Implement logging and reporting mechanisms to capture and analyze errors and unexpected behavior.

## Autoprompting: Generate Diverse Few-Shot Examples for Any Application (Part 2)

### 4. Step-by-Step Experiment Plan (Continued):

- **Exploration:** Incorporate techniques such as code summarization, call graph analysis, and type inference to provide more contextual information to the agent. Specifically, in any code snippet, if there are other user-defined functions, retrieve their metadata and use it in the next iteration of prompt generation.
- **Store:** Utilize a vector database or other structured storage mechanism that supports efficient retrieval and querying for storing few-shot examples and their outputs. Incorporate mechanisms for versioning and updating the stored data as the codebase and the underlying models evolve.
- **Experiments:** Once few-shot examples for different functionalities and their compositions are obtained, simulate different users with various intents and calculate goal completion and error rates using different models. Initially, start with a strong model, and once few-shot examples are available, test with weaker and open-source models.

### 5. Test Case Examples: Select a toy application from GitHub implemented in Python or JavaScript.

- **Direct prompting:** Provide the few-shot examples created and check the goal completion and error rates for the following scenarios.
- **Toy example:** Calculator app and different utterances to try.
  - Provide a complete user utterance with no ambiguity. For example:
    - \* Can you add 4 to 8.
    - \* Divide 6 by 9 and multiply it by 6.
  - Provide a user utterance with some ambiguity. For example:
    - \* Take 6 and 9, add them, and then subtract 8. Also, add 2 to the first one. – here the "first" one is ambiguous as it could be 6 or the intermediate answer ( $6+9=15$ ).
  - Provide a user utterance that is not related to the function. For example:
    - \* Please add A and J. The correct result would be refusing to answer instead of generating `add("A", "J")`.

**6. Fallback Plan:** If the proposed methodology does not yield satisfactory results, there are several areas to investigate. First, examine the documentation to ensure it adequately explains the basic functionality of each function. Then, assess the coding style to confirm it aligns with recommended practices. Subsequently, evaluate each module separately. For the NL module, verify that the examples are diverse and that the generated test cases are aligned. For the execution module, ensure that the correct error messages are being passed and explore ways to enhance them. The exploration module is the most challenging aspect; if any function has a high dependency on other functions, traversing it will be difficult. Therefore, initially focus on examples with limited to no function dependency and gradually increase the complexity.